

프로그래머를 위한 범주론

with) Haskell

김학재

2026.06.30

목차

1. 범주론
2. 패러다임
3. 부작용이 있는 함수
4. 현대 소프트웨어
5. 프로그래밍에서의 범주론

범주론이란

Category Theory (범주론) : 수학의 구조와 구조사이의 관계를 연구하는 학문.

- 수학에서의 범주론

수학에는 집합, 벡터공간 등 다양한 대상이 있다. 범주론은 이 대상 자체보다

대상 사이의 변환 (사상, morphism)

ex) $f(x) = 2x$

변환을 이어 붙이는 방법 (합성, composition)

ex) $(g \circ f)(x)$, $g(f(x))$

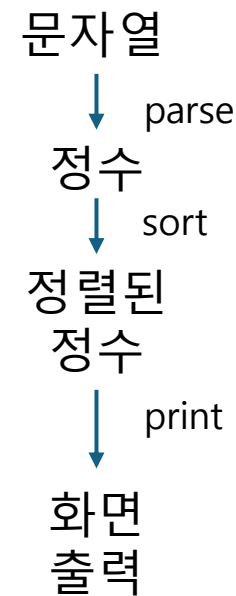
즉, '무엇'인지보다 '어떻게 연결'되는지를 연구하는 수학.

- 프로그래머에게 범주론

오른쪽과 같은 과정은 함수를 이어 붙이는 것이고

$\text{print} \circ \text{sort} \circ \text{parse}$ 이런식으로 표현 가능하다.

함수를 안전하게 합성하고 프로그램을 조립하는 방법을 설명하는 이론.

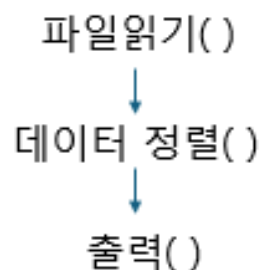


범주론이 프로그래머에게 왜 중요한가?

```
int add(int a, int b) {  
    return a + b;  
}
```

서브루틴 : 하나의 기능을 수행하도록 따로 만들어 놓은 코드 덩어리
ex) 함수

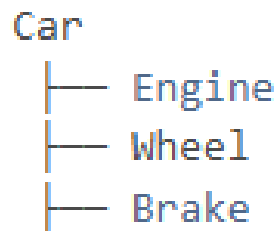
프로그래밍의 발전은 합성을 더 잘하게 된 역사이다.



— 서브루틴 합성,

코드블록 합성(구조적 프로그래밍), —

```
if (...) {  
    ...  
}  
while (...) {  
    ...  
}
```



— 객체 합성(객체 지향),

함수 합성 (함수형 프로그래밍) —

```
f(x) = x + 1  
g(x) = x * 2  
g(f(x))
```

즉, 프로그래밍은 작은 부품을 만들어 놓고 그것들을 조합해서 큰 프로그램을 만드는 방향으로 발전해 왔으며, 범주론은 이런 '합성' 자체를 수학적으로 연구하는 이론이라는 점을 강조한다.

간단한 Haskell 예시

Haskell을 생각을 표현하는 도구로 사용하면 좋다.

C++, Java -> 명령형 언어

Haskell -> 함수형 언어

명령형 언어 같은 경우 변수의 값을 계속 바꾼다.

```
int x = 1;  
x = x + 1;
```

반면 Haskell에서는 한 번 정한 값은 바꾸지 않는다.

이런 특징에서 수학의 함수와 비슷하다.

```
x = 1  
y = x + 1
```

```
vector<int> v = {1,2,3};
```

```
transform(  
    v.begin(),  
    v.end(),  
    v.begin(),  
    [](int x) {  
        return x + 1;  
    }  
);
```

map (+1) [1,2,3]

```
vector<int> v = {1,2,3};  
  
for (int &x : v) {  
    x = x + 1;  
}
```

패러다임

명령형 언어를 침범하는 새로운 함수형 기능

C, Java, C++에

람다, map, filter, reduce, immutable, stream 같은 함수형 기능들이 계속 추가되고 있다.

현대 프로그래밍에서 함수형 스타일이 필요하기 때문

저자는 여기서 '상전이 (Phase Transition)'을 이야기한다. *상전이 : 얼음 -> 물 -> 수증기

지금 프로그래밍 언어들이 조금씩 함수형 기능을 넣고 있지만

결국 프로그래밍 방식자체가 크게 바뀔것이다.

패러다임

왜 이런 변화가 생기는가? -> 멀티코어 CPU

과거에는 단일 코어 환경이 대부분이었고, 동작했기 때문에 객체지향에서도 동시성 문제가 크게 부각되지 않았다.

지금은 여러 코어가 동시에 실행되므로 같은 데이터를 접근하는 경우가 많아져 경쟁 상태와 같은 문제가 중요해졌다.

또 객체지향은 동시성에서 문제가 있다. 문제해결을 위해 뮤텝스 같은 것을 사용한다고 쳤을 때 lock이 객체 내부의 private 멤버로 캡슐화 되어있는 경우가 많아 어떤 객체가 어떤 lock을 획득하고 있는지 파악하기 어렵다.

따라서 교착 상태 발생시 원인 분석 및 디버깅하기가 어렵다는 것이다.

부작용이 있는 함수

순수함수, 부작용이 있는 함수.

순수 함수 : 어디에도 영향을 주지 않는다.

```
int add(int a, int b) {  
    return a + b;  
}
```

부작용이 있는 함수 : 메모리를 수정한다. (밖의 상태를 바꾸는 것)

ex) 파일을 바꾼다, 비밀번호를 바꾼다.

```
void WriteFile(string s)
```

```
void SetPassword(string pw)
```

-> 이 부작용들이 문제가 되는건 아니다. 당연히 파일을 쓰고, 비밀번호를 바꿀 수 있어야한다.

위의 WriteFile이나 SetPassword는 딱보면 무엇을하는 함수인지 알 수 있다.

*문제는 합성할 때이다.

숨겨진 부작용

부작용이 있는 함수들을 합성하기 시작하면

예를 들어

```
double price = getPrice();
```

함수 이름만 보면 가격 조회이다.

하지만 실제로는 전역변수 변경, 로그작성, 캐시 수정등을 하게된다.

```
double getPrice() {  
    accessCount++;  
    logAccess();  
    refreshCache();  
    ...  
}
```

이것이 숨겨진 부작용이고, 사용할때마다 프로그램 상태가 바뀌고 있기 때문에 버그가 생겨도 찾기가 매우 어려워진다는 것이다.

부작용은 확장되지 않는다.

여기서 확장된다. (scale) 이라는 것은
프로그램이 커져도 관리하기 쉽다. 라는 뜻이다.

프로그램 확장시 작은 프로그램은 문제가 안될지 몰라도,
거대한 프로그램에서는

각 함수들이 전역변수수정, 파일수정, 네트워크 요청, 로그 작성, 캐시 수정, DB수정 등 누가 무엇을 바꾸는지 아무도 모르게 된다.

예를 들어 money를 수정하는 함수가 있다면, 그 함수가 또 Deposit(), Withdraw(), 등 많은 곳에 들어있게되면
버그 추적하기 어려워진다.

부작용은 프로그램이 커질수록 관리하기 어려워진다.

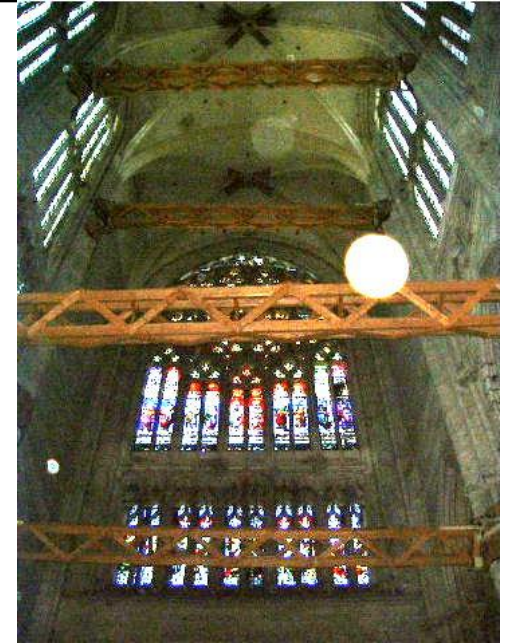
현대 소프트웨어

현대 소프트웨어는 중세 유럽의 고딕 성당과 비슷하다.
당시 건축가들은 더 높게, 더 크게, 더 가볍게 짓고 싶었다.
그런데 재료공학, 구조역학, 컴퓨터시뮬레이션도 없었던
당시 방법은 (짓는다 -> 무너진다 -> 기둥추가 -> 또 무너짐 -> 쇠막대 추가)
이런식으로 계속 무한 보강하는식으로 지었다.

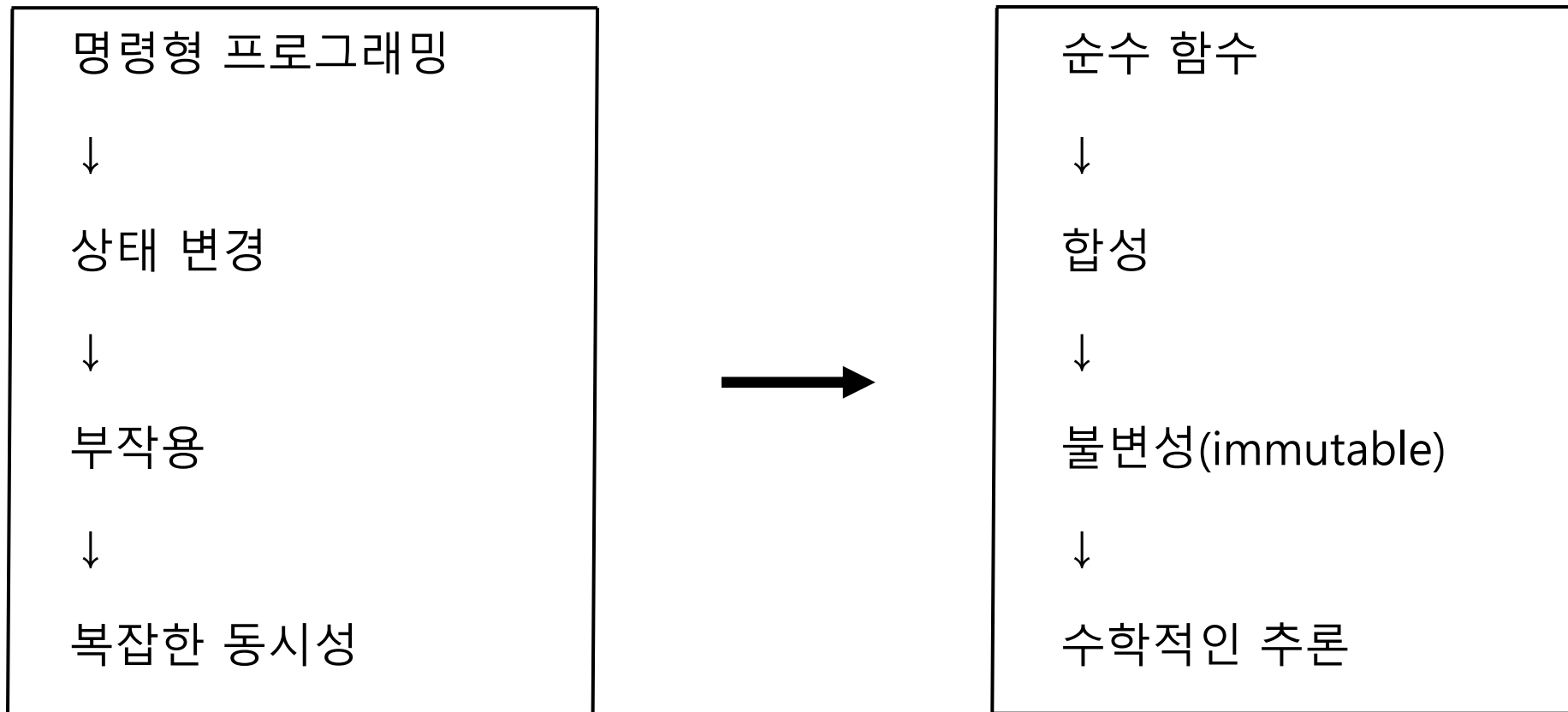
근데 현대소프트웨어도 비슷하다는 것이다.

현대에 운영체제, 웹 서버, 브라우저, 인터넷, 클라우드 같은 엄청난 시스템의 기초에는
전역변수, 부작용, 복잡한 상태변경, 복잡한 동시성 같은 관리하기 어려운 것들이 많이 깔려있다.

겉으로는 대단한 것이 있지만 기초는 그때그때 버티도록 덧대고 보강한 구조에 가깝다는 것이다.



기초를 고쳐야한다



부작용 자체는 피할 수 없지만, 프로그램이 커질수록 숨겨진 부작용은 관리하기 어려워지고 버그의 원인이 된다. 지금까지는 임시방편으로 계속 보완해오면서 거대한 소프트웨어를 만들어 왔지만,

앞으로 더 큰 시스템을 안정적으로 만들기 위해 범주론과 함수형 프로그래밍처럼 수학적으로 탄탄한 기초 위에서 소프트웨어를 설계해야한다.

범주(Category)는 무엇인가?

범주는 객체(Object)와 화살표(Arrow)로 이루어진다.

$A \longrightarrow B$

'객체가 무엇인가' 보다는 '객체들이 어떻게 연결되는가'가 중요하다.

화살표를 함수라고 생각하면 편하다.

$\text{int} \longrightarrow \text{string}$

```
string toString(int x);
```

입력타입 -> 출력타입을 나타낸다.

$A \xrightarrow{f} B$

:

A 타입을 입력받아 B 타입을 반환하는 함수 f

범주의 핵심은 '합성' 이다.

$A \xrightarrow{f} B \xrightarrow{g} C$

- >

$A \xrightarrow{h} C$

$h = g \circ f$

범주 정의

```
int doubleValue(int x) {  
    return x * 2;  
}
```

```
string toString(int x) {  
    return to_string(x);  
}
```



```
string h(int x) {  
    return toString(doubleValue(x));  
}
```

int —doubleValue—► int —toString—► string

int —h—► string

합성 : $A \xrightarrow{f} B \xrightarrow{g} C$ 있으면

$A \xrightarrow{g \circ f} C$ 존재해야한다.

항등사상 존재 : $A \xrightarrow{id} A$

1. **합성**할 수 있어야하고, **결합성** 만족

$$h \circ (g \circ f) = (h \circ g) \circ f = h \circ g \circ f$$

2. 모든 객체에는 **항등사상** 존재

$$A \xrightarrow{id} A$$

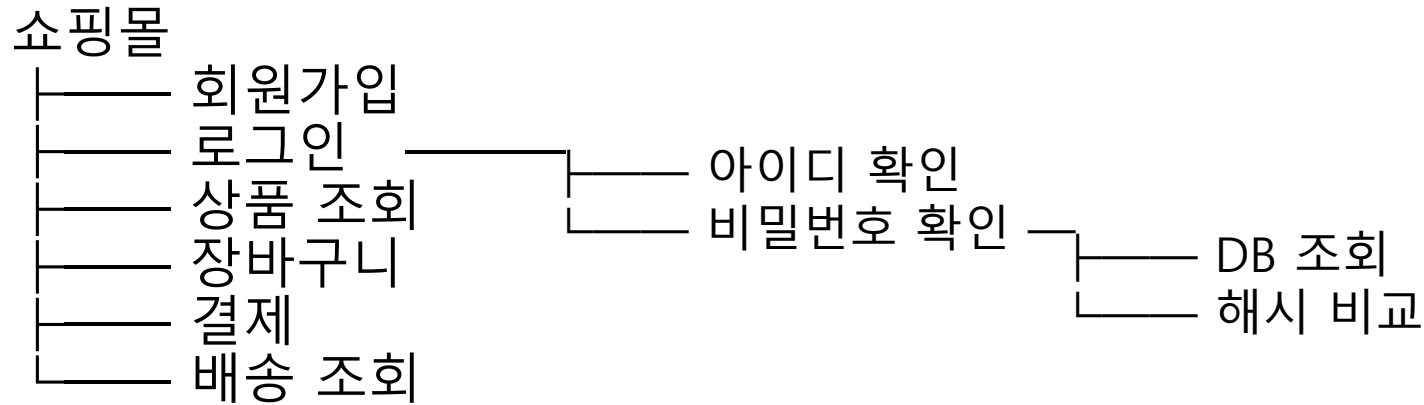
$$f \circ id = f$$

$$id \circ f = f$$

프로그래밍이란 무엇일까?

보통 우리는 프로그래밍 = 코드작성 이라고 생각할지도 모른다.
하지만 저자는 프로그래밍 = 문제 해결 이라고 생각한다.

예를 들어 온라인 쇼핑몰을 만들라고 하면 이처럼 계속 나뉘진다. (이것이 분해이다.)



마지막에는 결국 연결해야한다. (이것이 합성이다.)

우리가 분해하는 이유는 편하게 사람이 프로그래밍 하고, 관리하기 위해서이다 (객체지향의 이유)

프로그래밍에서의 범주론

```
class Stack
{
public:
    void push();
    void pop();
};
```

여기서 사람은 push만 알면되지 push안의 많은 줄의 코드들은 알 필요 없다.

예를들어 Haskell에서는 `sort :: [Int] -> [Int]` 이걸보고 정렬하는 함수구나 라는 것만 알면된다. 내부가 퀵소트인지 병합정렬인지는 알필요 없다.

그런데 범주론은 더 극단적이다.

범주론은 사실 저 `[Int] -> [Int]` 도 중요하게 보지않는다. `A->A`로 본다.

범주론은 어떻게 연결되는가만 본다는 것이다.

객체는 추상적이고 모호하다라고 보며 객체 내부는 관심이 없다.

프로그래밍에서의 범주론

그래서 프로그래머가 내부를 모르면 말이되나 라는생각이 들었다.

-> 객체를 합성하기 위해 내부구현을 알아야한다면 객체 지향의 장점을 잃는것이다.

저자가 말하는 좋은 프로그램은 작은 부품으로 분해할 수 있을 뿐 아니라, 그 부품의 내부 구현을 몰라도 다시 조립(합성) 할 수 있어야 한다는 것이다.

사람의 뇌는 복잡한 구현을 한꺼번에 처리할 수 없기 때문에, 우리는 각 부품을 "무엇을 하는지 (인터페이스)"만 알고 사용해야한다.

범주론은 이 생각을 극단으로 밀어붙여 객체 내부는 완전히 무시하고, 오직 객체들이 어떤 화살표(함수관계)로 연결되고 합성되는지만 연구하는 이론이다.

이것이 범주론이 프로그래밍과 깊게 연결되는 이유이다.

감사합니다